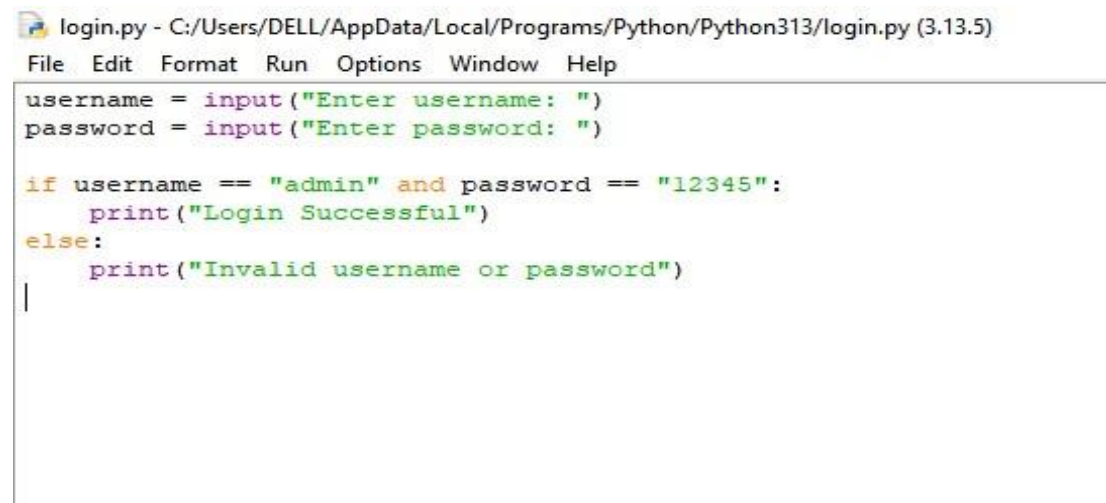


TASK 3: Secure Coding Review Report

Objective:

The objective of this task is to review a Python login application, identify security vulnerabilities, and implement security improvements by following secure coding practices.

Original Python Code:

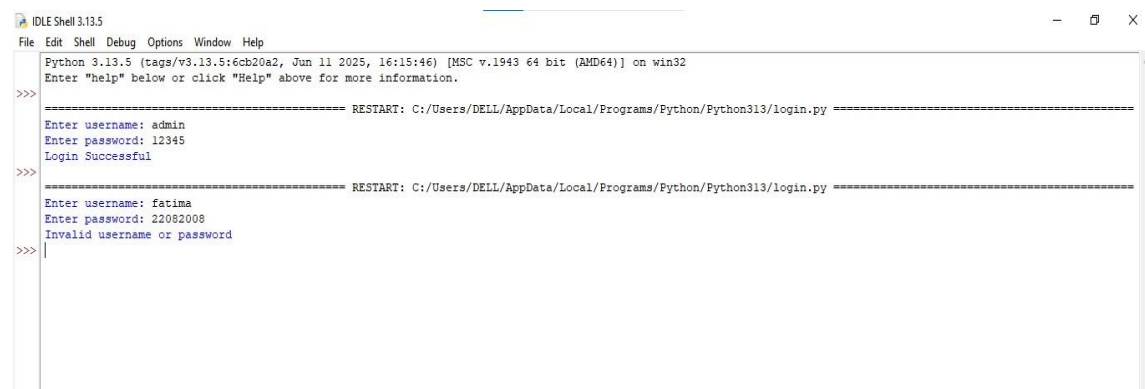


```
login.py - C:/Users/DELL/AppData/Local/Programs/Python/Python313/login.py (3.13.5)
File Edit Format Run Options Window Help

username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin" and password == "12345":
    print("Login Successful")
else:
    print("Invalid username or password")
```

Output of Original Code:



```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/DELL/AppData/Local/Programs/Python/Python313/login.py =====
Enter username: admin
Enter password: 12345
Login Successful

>>>
===== RESTART: C:/Users/DELL/AppData/Local/Programs/Python/Python313/login.py =====
Enter username: fatima
Enter password: 22082008
Invalid username or password

>>>
```

Security Vulnerabilities Identified:

During the code review, the following security issues were found:

- The username and password were hardcoded in the source code.
- The password was weak and easy to guess.
- Passwords were stored in plain text.
- There was no limit on login attempts, making the application vulnerable to brute-force attacks.

Modified Secure Python Code:

```
securelogin.py - C:/Users/DELL/AppData/Local/Programs/Python/Python313/securelogin.py (3.13.5)
File Edit Format Run Options Window Help

correct_username = "admin"
correct_password = "StrongPass123"

attempts = 3

while attempts > 0:
    username = input("Enter username: ")
    password = input("Enter password: ")

    if username == correct_username and password == correct_password:
        print("Login Successful")
        break
    else:
        attempts -= 1
        print("Invalid username or password.")
        print("Attempts left:", attempts)

if attempts == 0:
    print("Account locked. Too many failed login attempts.")
```

Output of Modified Code:

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:/Users/DELL/AppData/Local/Programs/Python/Python313/securelogin.py
Enter username: admin
Enter password: 12345
Invalid username or password.
Attempts left: 2
Enter username: admin
Enter password: StrongPass123
Login Successful
>>> |
```

Improvements Made:

The following improvements were implemented:

- Replaced the weak password with a stronger password.
- Added a limit of three login attempts.
- Locked the account after three failed login attempts.
- Improved the overall security of the login process.

Recommendations:

- Use strong and unique passwords.
- Store passwords securely using hashing algorithms such as bcrypt or SHA-256.
- Avoid hardcoding credentials in source code.
- Store sensitive information in secure databases or configuration files.
- Implement account lockout and multi-factor authentication where possible.

Conclusion:

This secure coding review identified several common security vulnerabilities in the original Python login application. After implementing the recommended improvements, the application became more resistant to brute-force attacks and followed better security practices. This task helped improve understanding of secure coding principles and software security.